

Recursive Insertion Sort

CSD2183 Data Structures — Week 01 Lab — 2 September 2026

Start here

Implement insertion sort recursively: to sort the first n keys, sort the first $n - 1$ recursively, then insert the last key into that sorted prefix.

1. Write it in `recursive_insertion_sort.cpp` — the only file you change, and the only one you submit.
2. Build with `make`. Check your work with `make test`.
3. Submit to Gradescope **before** you end the exam in Examena.

Keep this PDF open: once you follow the link to Gradescope you cannot return to xSITE, and general internet access is restricted while Examena runs.

You have **45 minutes**. Work independently, but feel free to ask your instructors for clarification of the concepts or for help with C++. You may use the files in this archive, your local development tools, and any offline reference material, including notes, textbooks, and locally stored documentation. Do not use generative AI or other AI-assisted coding tools, whether online or installed locally. **Nothing in today's lab is graded**; the exercise gives you practice with the lab workflow before it is used for graded work. The sections below are reference — start with the task, and come back to them as you need.

1 Your task

Implement insertion sort **recursively** in C++, by completing this function in `recursive_insertion_sort.cpp`:

```
void recursiveInsertionSort(std::vector<int> &ar, std::size_t n);
```

The contract. `ar` is a mutable vector of integer keys and `n` is how many elements at its front are to be sorted, with $0 \leq n \leq \text{ar.size}()$. After the call, `ar[0 .. n-1]` is in ascending order. The vector is sorted in place, so the function returns nothing and elements from index `n` onwards must not be touched. `n` equal to 0 or 1 is legitimate and must work. The signature must not change.

Note that `n` is a **length**, not an index: the last key of the prefix you are sorting is `ar[n - 1]`, not `ar[n]`.

Your implementation must call itself, and the algorithm must be yours. It is the *outer* repetition — growing the sorted prefix one key at a time — that must become recursion. The insertion step may still use a loop; shifting the hole is naturally written as a **while**. An implementation that loops over the prefixes instead, or that hands the work to `std::sort`, is not what this exercise asks for. The autograder checks for a recursive call and rejects library sorts.

Modify only `recursive_insertion_sort.cpp`. Everything else is supplied: the header, `main.cpp`, the input helper, the `makefile`, the test driver, and `tests/` with paired `input_*.txt` and `expected_*.txt` files. Only that one file is submitted, so any change elsewhere is discarded — and editing an `expected_*.txt` would make a test pass without fixing anything.

2 Insertion sort

The iterative algorithm from Monday's lecture:

Input: *ar*, array of keys

Output: *ar*, sorted in ascending order

```
1: n_keys ← length(ar)  
   ▷ Loop invariant, before each iteration: ar[0...i − 1] is sorted  
2: for i ← 1 ... n_keys − 1 do  
3:   active_key ← ar[i]  
4:   h ← i                                     ▷ hole index  
5:   while h > 0 and ar[h − 1] > active_key do  
6:     ar[h] ← ar[h − 1]                       ▷ shift right  
7:     h ← h − 1  
8:   ar[h] ← active_key
```

3 The recursive reformulation

The iterative version grows a sorted prefix by one key on each pass through the outer **for** loop. Recursion replaces that outer loop:

To sort the first *n* keys, first recursively sort the first *n* − 1, then insert the last key into that sorted prefix.

Use the same hole-shifting idea for the insertion step. The base case, the recursive call and the C++ are yours to work out.

4 Building and testing

An input file holds integers separated by commas, for example 31, 41, 59, 26, 41, 58, and the program prints the sorted keys in the same format. An empty input prints an empty line, duplicates are kept, and values span the full range of `int`.

Open a terminal in the folder holding the `makefile`, then:

Command	Effect
<code>make</code>	compile the program
<code>make test</code>	run every supplied case and compare against the expected output
<code>make run FILE=input_reverse_sorted.txt</code>	run one input and print the result
<code>make clean</code>	delete the compiled files and start fresh

`make test` checks that your implementation is recursive, runs every case in `tests/`, and checks that sorting a prefix leaves the rest of the vector alone. It reports **PASS** or **FAIL** for each check without stopping at the first failure. `make run` takes a name from `tests/`, or a path to a file of your own.

If `make test` stops with a crash or **Segmentation fault** instead of a list of results, the usual cause is a missing base case, so the recursion never ends.

The supplied code compiles before you write anything, leaving the vector unsorted, so most cases fail until you implement the function. When any check fails, `make test` ends with `make: *** [test] Error 1` — that is the failure count, not a broken build.

The ten cases run from an empty file and a single key through sorted, reverse-sorted and all-equal arrays to duplicates, the extremes of `int`, and a thousand random keys. Cases you add yourself are listed as `SKIP` until you supply a matching `expected_*.txt`. Passing every check is a strong indication that your implementation works locally; Gradescope runs further checks. Use `make test` as you work rather than saving all testing until the end.

5 Submitting

Follow the Gradescope link in the xSITE quiz. It opens **Week 01 Lab: Recursive Insertion Sort** — if you find yourself elsewhere in Gradescope, that is the assignment to submit to. Upload **only** `recursive_insertion_sort.cpp` — not the archive, the other source files, or the compiled program. Nothing is uploaded to xSITE. You may submit **as often as you like**; the most recent submission is the one that counts, so once you have something working, do not overwrite it with an experiment.

- **Submit even if your code is incomplete or fails some tests.** What is recorded is your independent attempt, and a partial attempt is a legitimate result.
- **Submit to Gradescope before you end the exam in Examena.** What you submit while Examena is running is your independent attempt; anything submitted later is practice, not part of that attempt.

If something goes wrong, raise your hand and tell an instructor. Do not restart your laptop or leave the room to look for help.

If you finish early

Optional, and only once your working solution is safely submitted. How much work does insertion sort actually do? Count two things:

- **comparisons with the active key** — each evaluation of `ar[h - 1] > active_key`; and
- **right shifts** — each time `ar[h] = ar[h - 1]` moves a key one position right.

The counting happens inside `recursiveInsertionSort`, but the totals have to be printed from `main.cpp`, so you will need a way to carry them out of the function — extra reference parameters, a couple of file-scope variables, or a small helper of your own. **For this challenge only, and only after you have submitted, you may change the function signature, the header and `main.cpp`.** `make test` stops working once you do; `make run` still works. Copy the whole project folder first and work in the copy — the copy is what keeps your submitted version intact. Do not upload anything from the challenge.

Three supplied inputs hold **eight keys each**: `input_already_sorted.txt`, `input_reverse_sorted.txt` and `input_all_equal.txt`. Being the same size, they isolate the effect of the *arrangement* of the input.

1. Before you run anything, **predict**. Which of the three produces the most comparisons, and which the fewest? What about shifts? Write your predictions down.
2. Run all three and measure.
3. At least one of the three results surprises most people. Be ready to say why in the discussion at the end of the lab.

One warning: a counter inside the `while` body counts only the comparisons that came out true. Getting it exactly right, given that `&&` short-circuits, is part of the challenge.